the same program; the results differ completely even if there is a small variation.

```
print(hash("Lorem"))
print(hash("Lorem"))
print(hash("Loren"))

5675358576591581973
5675358576591581973
1733253194914647111
```

In general, each rerun will produce random values (as given below). This is expected behaviour and is used as a countermeasure against Denial-of-Service (DoS) or hash flooding. Hash flooding occurs when the attacker knows that for the list of keys the hash function returns the same hash bucket, causing a collision. If the language does not provide a randomised hash function or the application server does not recognise attacks using multi-collisions, an attacker can degenerate the hash table by sending lots of colliding keys. The algorithmic complexity of inserting 'n' elements into the table then goes to O(n**2), making it possible to exhaust hours of CPU time using a single HTTP request (see *https://ocert.org/advisories/ocert-2011-003.html).*

```
>> /usr/bin/python3 -c 'print(hash("Lorem"))'
3856737851614672944
>> /usr/bin/python3 -c 'print(hash("Lorem"))'
3400600052546168698
>> /usr/bin/python3 -c 'print(hash("Lorem"))'
3757617178999059249
```

At the time of writing this, the siphash24 algorithm was used to generate hash by built-in hash. This was designed to prevent DoS hash flooding. Along with SipHash, random salt was used.

In Python 3.3, a random salt value is included when computing hash codes for str, bytes, and datetime objects, as documented in issue 13703 (hash collision security issue). The salt value is constant within a Python process but varies between interpreter runs. With PEP-456, Python 3.4 adopted the SipHash cryptographic function to compute hash codes for str and bytes objects. The random salt and SipHash are security measures to prevent DoS attacks. Check the reference for Python 3.8.9 at *https://en.wikipedia.org/wiki/SipHash.*

```
import sys
print(sys.hash_info.algorithm)
siphash24
```

To disable hash randomisation by setting a fixed seed value through the PYTHONHASHSEED environment variable, use the following:

```
>> PYTHONHASHSEED=1 /usr/bin/python3 -c
'print(hash("Lorem"))'
440669153173126140
>> PYTHONHASHSEED=1 /usr/bin/python3 -c
'print(hash("Lorem"))'
440669153173126140
>> PYTHONHASHSEED=1 /usr/bin/python3 -c
'print(hash("Lorem"))'
440669153173126140
```

Instances of built-in mutable types—like lists, sets, and dicts—aren't hashable. Only immutable objects are hashable (keys alone).

```
>> /usr/bin/python3 -c 'hash([1, 2, 3])'
Traceback (most recent call last):
 File "<string>", line 1, in <module>
TypeError: unhashable type: 'list'
```

Small integers of hash values are equal to themselves, which is an implementation detail that CPython uses for simplicity and efficiency. The hash values don't matter as long as you can calculate them in a deterministic way.

```
print(hash(5))
print(hash(500000))
print(hash(50000000000))
print(hash(500000000000000000000))
print(hash(5000000000000000000000000000000))
print(hash(5000000000000000000000000000000000000000))

Output:
5
500000
50000000000
1937910009842106584
20450418580029579
•    24958392007006097
    •  >>>[hash(i) for i in range(4)]
       [0, 1, 2, 3]

       Comment: cpython implementation dictobject.c
```

This isn't necessarily bad! To the contrary, in a table of size 2**i, taking the low-order i bits as the initial table index is extremely fast, and there are no collisions at all for dicts indexed by a contiguous range of ints. So, this gives better-than-random behaviour in common cases, and that's very desirable.